
Table of Contents

Devoir TP - Résolution d'un problème inverse : déconvolution d'un signal triangulaire	1
1. Construction du jeu de données	1
1.a Simulation du signal d'entrée	1
1.b Simulation du filtre	2
1.c Simulation du problème direct - première approche	3
1.d Simulation du problème direct par approche matricielle	6
2. Déconvolution	7
2.a Déconvolution naïve	7
2.b Déconvolution par moindres carrés	8
2.c Déconvolution par régularisation	9

Devoir TP - Résolution d'un problème inverse : déconvolution d'un signal triangulaire

Auteur : AZIOUIZ INASS Master Ingénierie des Systèmes Complexes M2 ISC 903.4 Optimisation numérique et sciences des données

```
clear all;  
close all;  
clc;
```

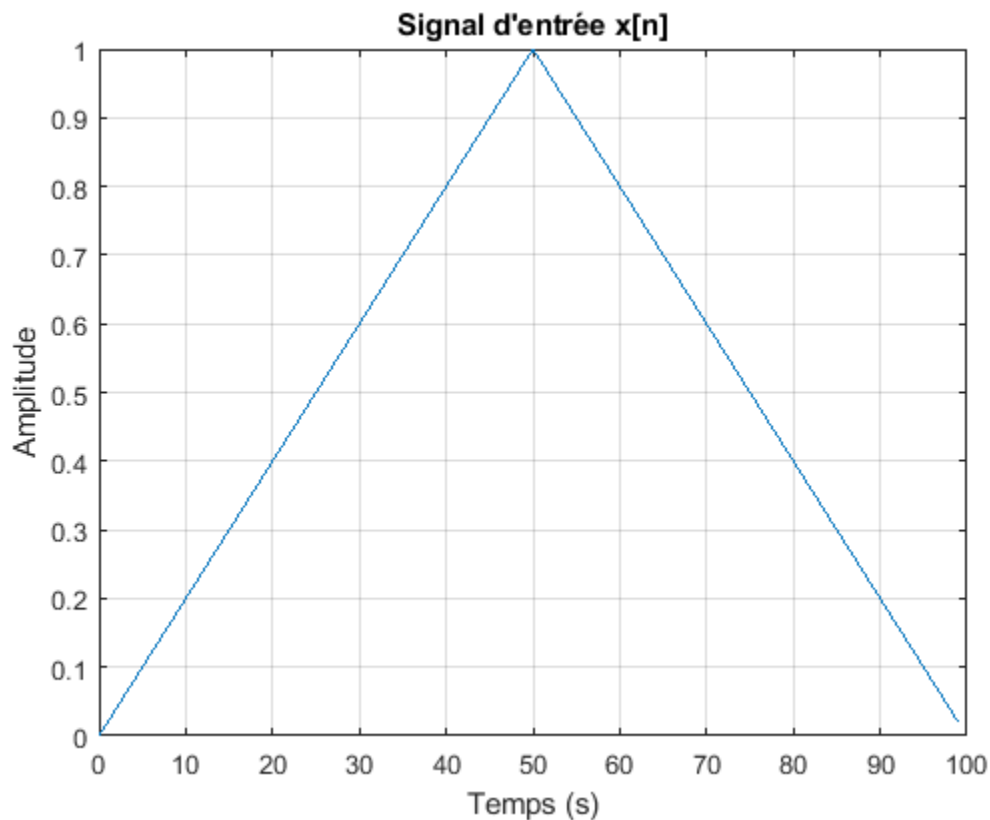
1. Construction du jeu de données

1.a Simulation du signal d'entrée

Cette section crée un signal triangulaire sur une durée de 100 secondes avec une fréquence d'échantillonnage de 1 Hz. Ce signal servira de base pour les simulations de convolution et de déconvolution ultérieures.

```
Lx = 100;           % Durée d'observation du signal d'entrée (en secondes)  
fe = 1;             % Fréquence d'échantillonnage (en Hz)  
Te = 1/fe;          % Période d'échantillonnage (en secondes)  
Nx = Lx/Te;         % Nombre d'échantillons du signal x  
nx = 0:Nx-1;        % Indice temporel (entier)  
tx = nx*Te;         % Base de temps (en secondes)  
  
% Création du signal triangulaire  
t1 = 0:Lx/2 - 1;  
x1 = t1/(Lx/2);  
  
t2 = Lx/2:Lx - 1;  
x2 = (-t2+Lx)/(Lx/2);  
  
x = [x1 x2];  
  
% Affichage du signal d'entrée  
figure;
```

```
plot(tx, x);  
title('Signal d\'entrée x[n]');  
xlabel('Temps (s)');  
ylabel('Amplitude');  
grid on;
```



1.b Simulation du filtre

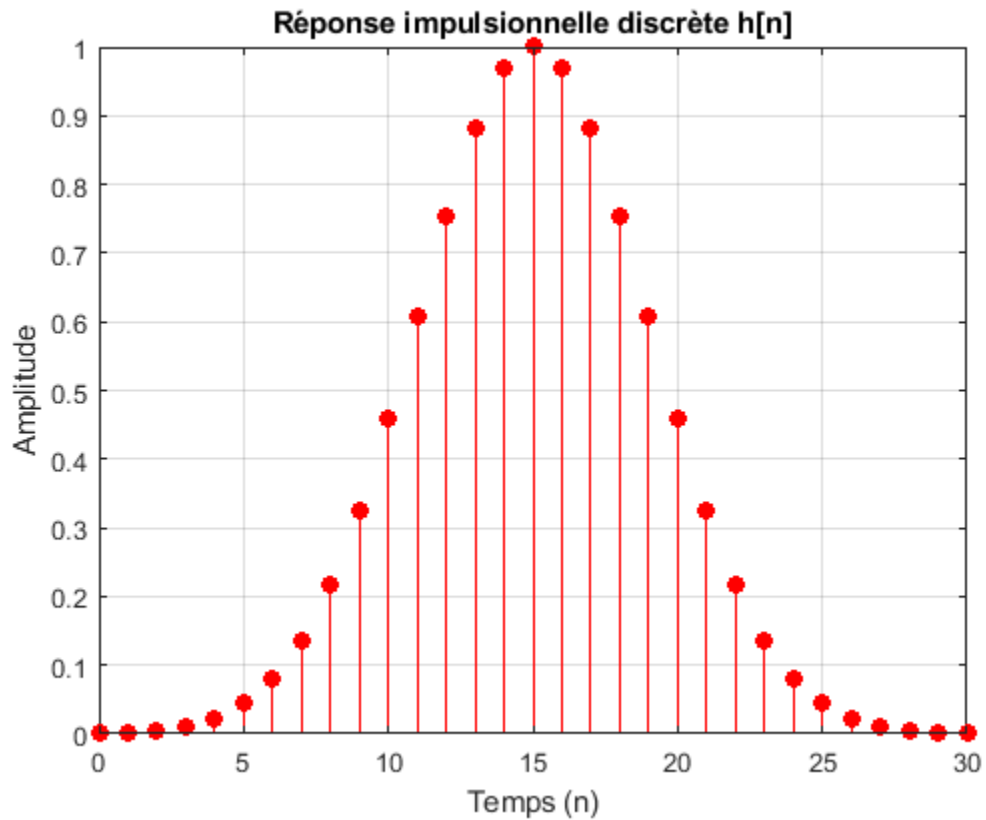
Cette partie simule la réponse impulsionnelle d'un filtre gaussien. Le filtre est défini par sa moyenne ($m = 15$) et son écart-type ($\sigma = 4$). La réponse est tronquée à $N_h = 30$ pour des raisons pratiques. Ce filtre sera utilisé pour convoluer le signal triangulaire.

```
% Paramètres du filtre gaussien  
mh = 15;           % Moyenne du filtre  
sigmah = 4;        % Écart-type  
Nh = 30;           % Durée de troncature  
  
n = 0:Nh; % Indices discrets de la réponse impulsionnelle  
  
% Calcul de la réponse impulsionnelle h[n]  
h_n = (1 / (sigmah * sqrt(2 * pi))) * exp(-((n - mh).^2) / (2 * sigmah^2));  
h_n = h_n / max(h_n); % Normalisation  
  
% Affichage de la réponse impulsionnelle  
figure;  
stem(n, h_n, 'filled', 'r');
```

```

title('Réponse impulsionnelle discrète h[n]');
xlabel('Temps (n)');
ylabel('Amplitude');
grid on;

```



1.c Simulation du problème direct - première approche

Ici, nous simulons le problème direct en convoluant le signal d'entrée avec le filtre gaussien. Nous ajoutons ensuite un bruit de quantification pour simuler les erreurs d'un convertisseur analogique-numérique.

```

% 1. Simulation du signal de sortie non bruité ynb[n]
ynb_1 = conv(x, h_n);

figure;
plot(ynb_1);
title('Signal de sortie non bruité ynb[n]');
xlabel('n');
ylabel('Amplitude');
grid on;

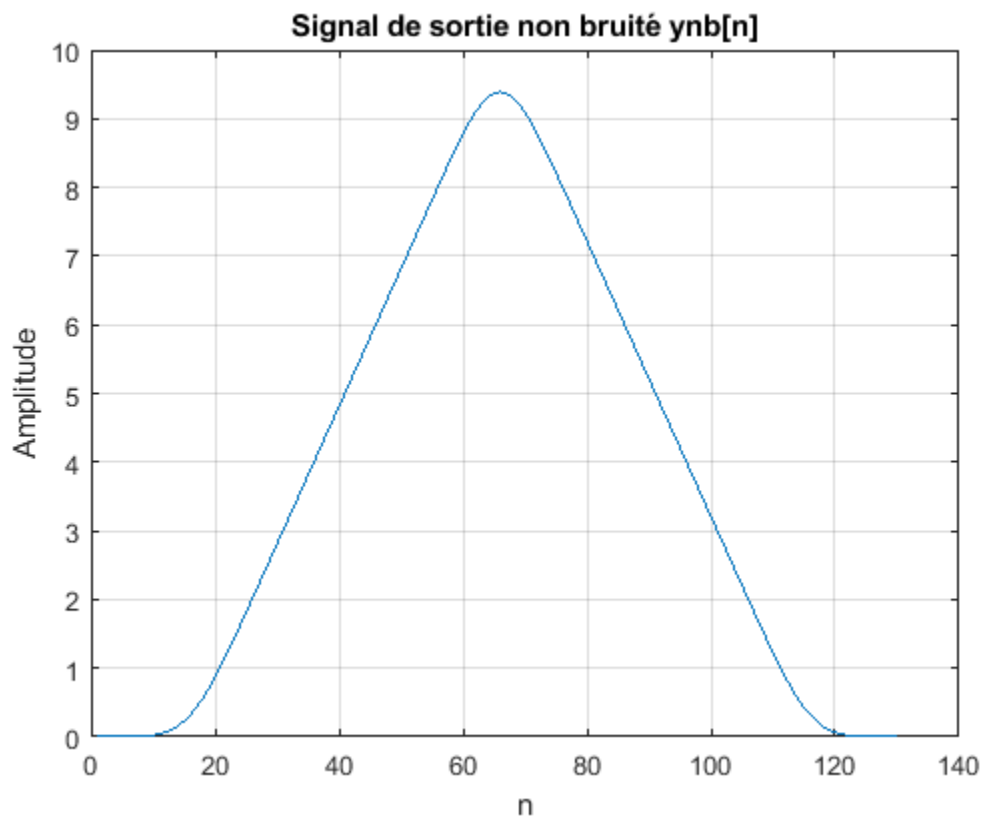
% 2. Simulation et représentation du signal quantifié y
% La quantification introduit un bruit artificiel pour simuler un
% scénario réaliste de mesure.
y = round(ynb_1 * 2^10) / 2^10; % Quantification

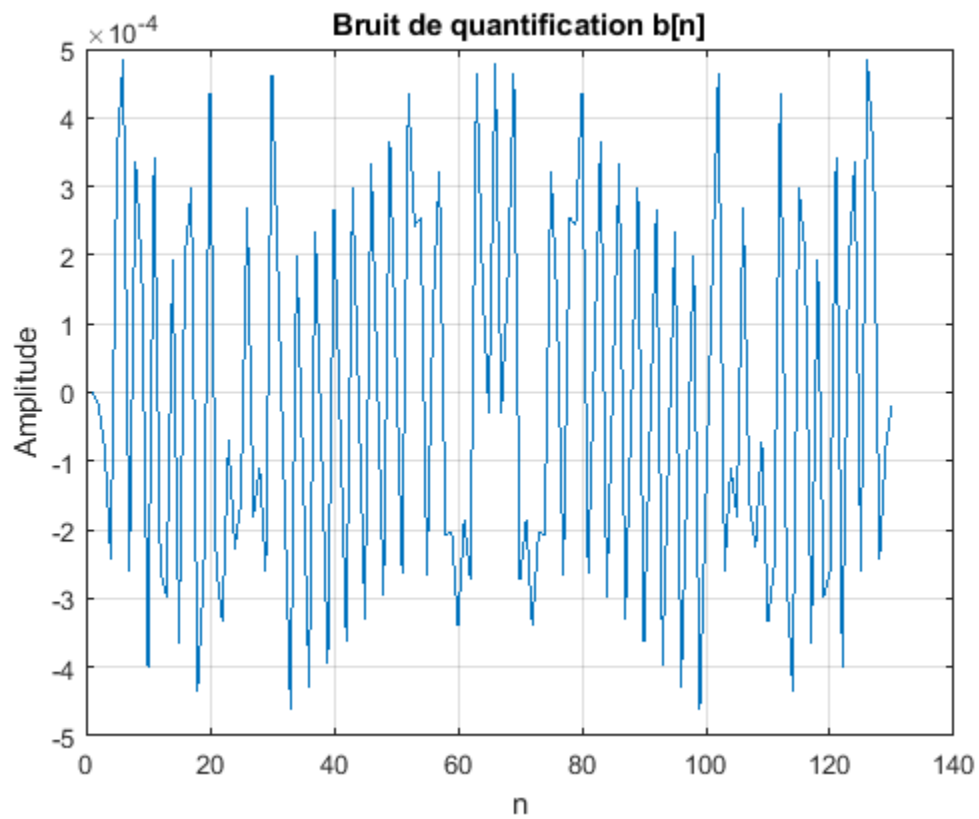
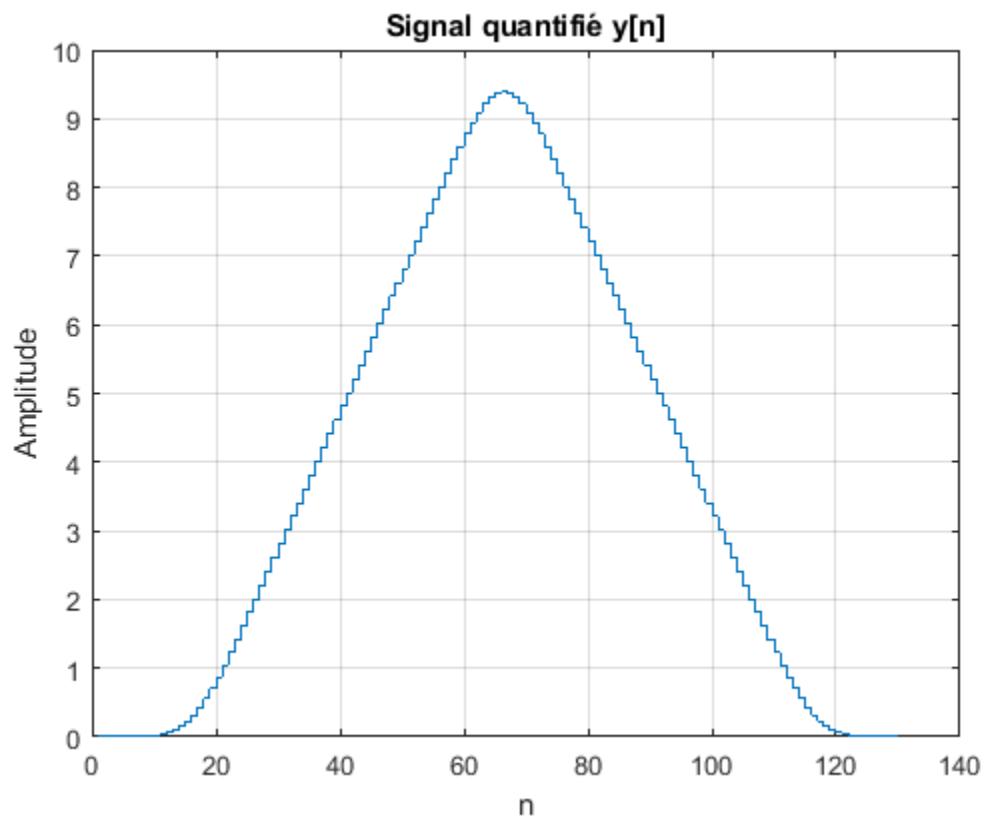
```

```
figure;
stairs(y);
title('Signal quantifié y[n]');
xlabel('n');
ylabel('Amplitude');
grid on;

% 3. Détermination et représentation du bruit de quantification b[n]
% Le bruit de quantification est la différence entre le signal non bruité
% et le signal quantifié.
b_n = y - ynb_1;

figure;
plot(b_n);
title('Bruit de quantification b[n]');
xlabel('n');
ylabel('Amplitude');
grid on;
```





1.d Simulation du problème direct par approche matricielle

Cette section réalise la même opération de convolution que précédemment, mais en utilisant une approche matricielle. Cette méthode est utile pour comprendre la structure des problèmes inverses.

```
% 1. Détermination de la dimension Ny du vecteur de sortie ynb
Ny = length(y nb_1);

% 2. Construction de la matrice de convolution H
H_ligne = zeros(1, Nx);
H_colonne = zeros(Ny, 1);
H = toeplitz(H_colonne, H_ligne);

for i = 1:Nx
    for j = 1:length(h_n)
        var = i + j - 1;
        H(var, i) = h_n(j);
    end
end

% 3. Détermination de la sortie non bruitée par opération matricielle
x_t = x';
y nb_2 = H * x_t;

figure;
plot(y nb_2);
title('Sortie non bruitée : approche matricielle');
xlabel('n');
ylabel('Amplitude');
grid on;
```



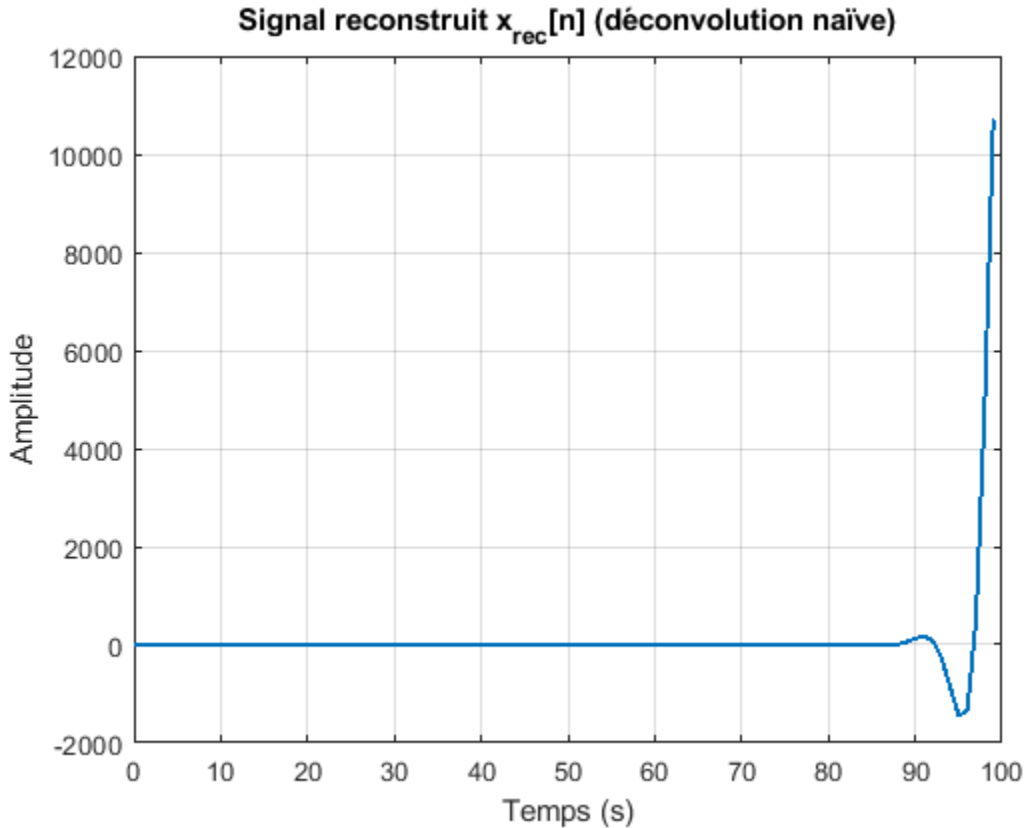
2. Déconvolution

2.a Déconvolution naïve

Cette approche utilise la fonction `deconv` de MATLAB pour tenter de récupérer le signal d'entrée à partir de la sortie filtrée. Cependant, cette méthode est sensible au bruit et peut donner des résultats instables.

```
% Reconstruction à partir du signal non bruité
[x_rec, ~] = deconv(ynb_1, h_n);

figure;
plot((0:length(x_rec)-1) * Te, x_rec, 'LineWidth', 1.5);
title('Signal reconstruit x_{rec}[n] (déconvolution naïve)');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
```



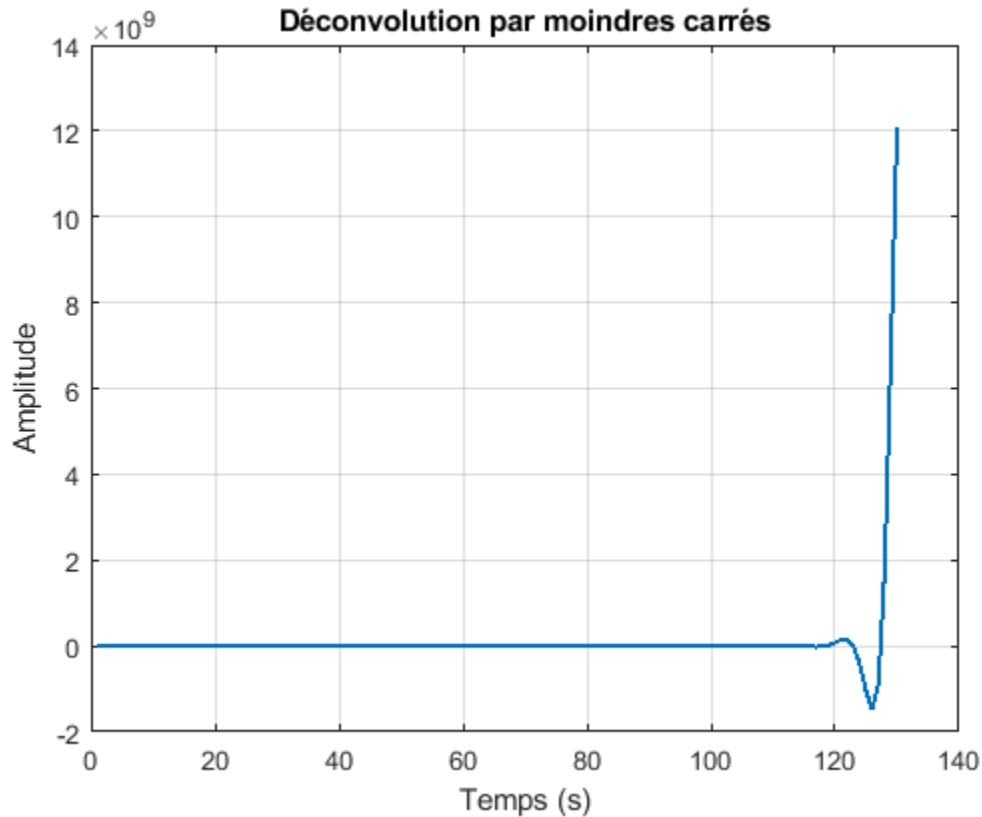
2.b Déconvolution par moindres carrés

Cette méthode résout le problème de déconvolution en utilisant l'approche des moindres carrés, qui est plus robuste face au bruit par rapport à la déconvolution naïve.

```
% Construction de la matrice H pour la déconvolution
H = toeplitz([h_n zeros(1, Nx-1)], [h_n(1) zeros(1, Ny-1)]);

% Résolution des moindres carrés
x_lsq = H \ ynb_1';

figure;
plot(x_lsq, 'LineWidth', 1.5);
title('Déconvolution par moindres carrés');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
```

2.c Déconvolution par régularisation

Ici, nous appliquons la régularisation pour stabiliser la déconvolution. Différentes valeurs de σ sont testées pour évaluer leur influence sur la qualité de la reconstruction du signal original.

```
sigma_values = [2, 4, 8];
reconstruction_errors = zeros(1, length(sigma_values));

figure;
for i = 1:length(sigma_values)
    sigma_h = sigma_values(i);
    h_n = (1 / (sigma_h * sqrt(2 * pi))) * exp(-(n - m_h).^2 / (2 *
sigma_h^2));
    h_n = h_n / max(h_n);

    [x_rec, ~] = deconv(y, h_n);
    tx_rec = (0:length(x_rec)-1) * Te;

    if length(x_rec) > length(x)
        x_rec = x_rec(1:length(x));
    elseif length(x_rec) < length(x)
        x_rec = [x_rec; zeros(length(x) - length(x_rec), 1)];
    end

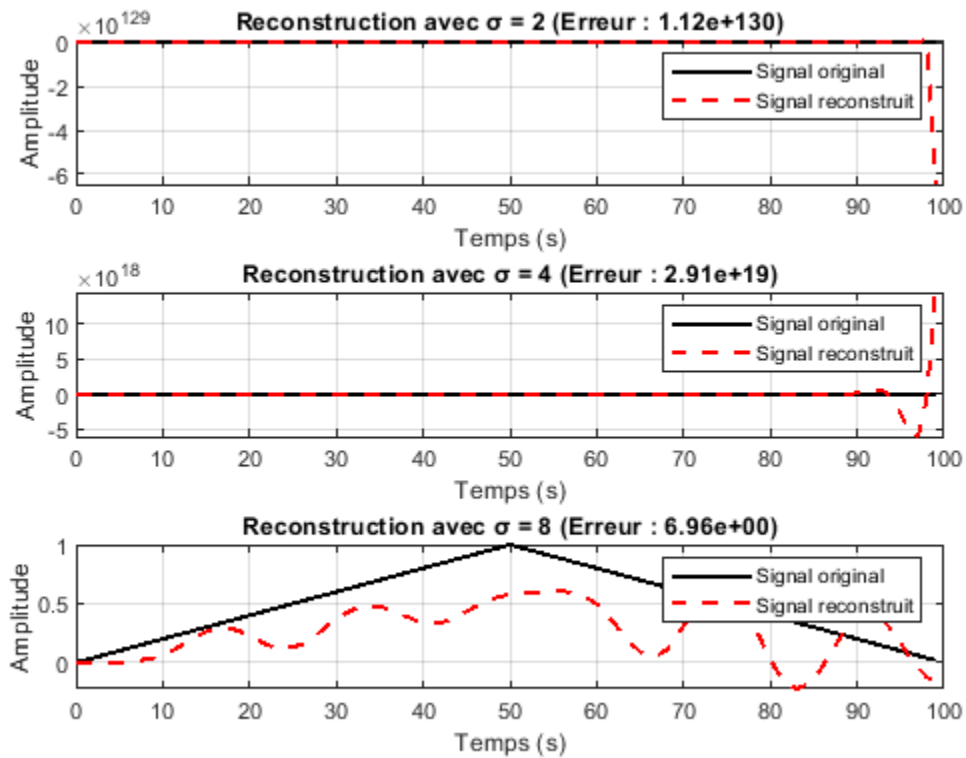
    reconstruction_error = norm(x - x_rec') / norm(x);
    reconstruction_errors(i) = reconstruction_error;
end
```

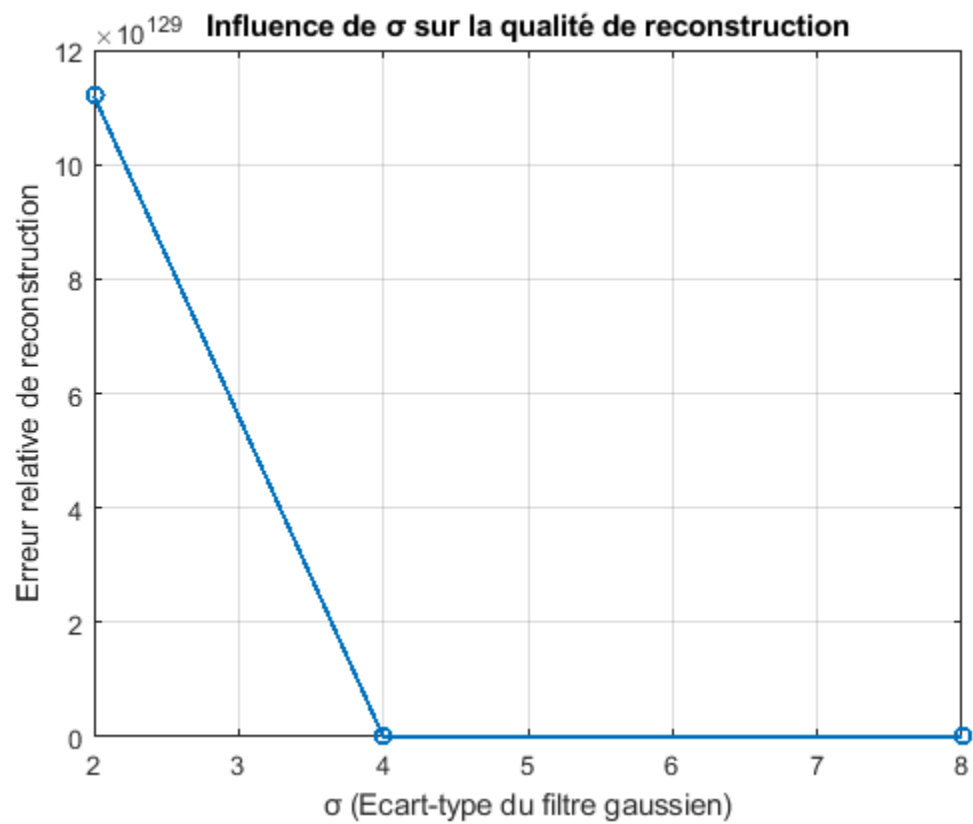
```

subplot(length(sigma_values), 1, i);
plot(tx, x, 'k', 'LineWidth', 1.5); hold on;
plot(tx_rec, x_rec, 'r--', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title(['Reconstruction avec  $\sigma =$ ', num2str(sigmah), ...
      ' (Erreur : ', num2str(reconstruction_error, '%.2e'), ')']);
legend('Signal original', 'Signal reconstruit');
grid on;
end

figure;
plot(sigma_values, reconstruction_errors, '-o', 'LineWidth', 1.5);
xlabel('σ (Ecart-type du filtre gaussien)');
ylabel('Erreur relative de reconstruction');
title('Influence de σ sur la qualité de reconstruction');
grid on;

```





Published with MATLAB® R2024a